Cliente, servidor y contenido dinámico

Una aplicación web reparte responsabilidades. El cliente presenta la interfaz e inicia peticiones; el servidor protege datos, aplica reglas y produce respuestas. Entender esa frontera evita dos errores frecuentes: confiar en el navegador para decisiones de seguridad y llamar "backend" a cualquier recurso descargado por una página.

Recorrido de una petición

Cuando se solicita una página dinámica:

1. El navegador resuelve el nombre de dominio y establece la conexión.
2. Envía una petición HTTP al servidor.
3. La infraestructura dirige la petición hacia la aplicación adecuada.
4. El código servidor valida la entrada, aplica reglas y consulta otros sistemas si es necesario.
5. El servidor devuelve una respuesta HTTP.
6. El navegador interpreta HTML, CSS y JavaScript y representa la interfaz.

La respuesta puede contener HTML completo, JSON, una imagen, un archivo o ningún contenido. HTTP no obliga a construir una API REST.

HTTP es el contrato de la conversación

HTTP es el protocolo que estructura esa conversación entre cliente y servidor. No es solo la URL ni el contenido que vemos en pantalla: cada intercambio tiene una petición y una respuesta, con metadatos que describen cómo interpretar lo que se envía y lo que se recibe.

Una petición HTTP mínima puede verse así:

```
GET /api/hello HTTP/1.1
Host: localhost:8080
Accept: application/json
```

El servidor responde con un estado, campos descriptivos y una representación:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"message":"Hola desde el servidor"}
```

La primera línea de la petición indica el método, el destino y la versión; las cabeceras aportan información adicional; la línea en blanco separa las cabeceras del contenido opcional. En la respuesta, `200 OK` indica el resultado y `Content-Type` permite interpretar el contenido. El navegador oculta buena parte de este intercambio al mostrar la página, por eso conviene observarlo directamente con HTTPie y con la pestaña **Red** de las herramientas de desarrollo.

En HTTP/2 y HTTP/3 la representación de los mensajes cambia, pero este contrato lógico se mantiene. La aplicación sigue teniendo que distinguir petición, respuesta, estado, cabeceras y contenido.

Qué se ejecuta en cada lado

Cliente	Servidor
Renderiza la interfaz	Aplica reglas de negocio
Gestiona interacción inmediata	Autoriza operaciones
Puede validar para mejorar la experiencia	Valida siempre los datos recibidos
Conserva solo secretos que sean públicos para ese usuario	Protege credenciales y claves privadas
No es una frontera de confianza	Es responsable de la decisión final

El usuario controla su navegador y puede modificar HTML, JavaScript y peticiones. Por eso una validación realizada solo en cliente nunca protege el sistema.

Estático no significa inerte

Un recurso es estático cuando el servidor entrega el mismo archivo almacenado sin generarlo para esa petición. Ese HTML puede ejecutar JavaScript, reaccionar al usuario y consultar servicios externos. Puede ser muy interactivo.

Un contenido es dinámico en servidor cuando la respuesta se genera o adapta al procesar la petición. Por ejemplo:

- mostrar las tareas del usuario autenticado;
- calcular un precio según reglas vigentes;
- representar una plantilla con datos de una base de datos;
- devolver JSON filtrado según permisos.

La diferencia importante no es si la pantalla se mueve, sino **dónde y cuándo se produce el contenido**.

Ventajas de ejecutar código en servidor

- Acceso controlado a bases de datos y sistemas internos.
- Aplicación centralizada de reglas y permisos.
- Protección de claves, credenciales y algoritmos privados.
- Respuestas adaptadas al estado persistente y a la identidad.
- Actualización de la lógica sin distribuir una aplicación nueva a cada cliente.

También tiene costes: infraestructura, latencia, concurrencia, observabilidad y una superficie de ataque que debe gestionarse.

Comprobación

Para una petición observada en las herramientas del navegador, identifica:

1. Qué datos conocía el cliente antes de enviarla.
2. Qué decisión solo podía tomar el servidor.
3. Qué parte de la respuesta interpreta el navegador.
4. Qué validación sería inseguro delegar exclusivamente al cliente.

Repite la observación contra el ejemplo Hello Server y compara `/`, `/api/hello` y `/health`. No describas solo lo que aparece en la pantalla: relaciona cada representación con su `Content-Type`, estado y código servidor.

Esta evidencia trabaja RA1.a y RA1.b.